

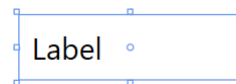


Steuerelemente

Label

- Dient zur Ausgabe von kurzen Texten

- XAML: `<Label></Label>`



- Wichtige Eigenschaften:
 - `Background`: Hintergrundfarbe
 - `Foreground`: Vordergrundfarbe (zum Beispiel Schriftfarbe)
 - `Content`: Beschriftung
 - `BorderBrush`: Rahmenfarbe
 - `BorderThickness`: Dicke des Rahmens
 - `HorizontalContentAlignment`: horizontale Position des Inhaltes
 - `VerticalContentAlignment`: vertikale Position des Inhaltes

Das 'Label'-Control ist ein grundlegendes Steuerelement in WPF, das zum Anzeigen von Text sowie beliebigen anderen Inhalten wie Controls oder Model-Objekten verwendet wird. Es bietet eine Vielzahl von Eigenschaften, mit denen der Textinhalt und das Aussehen des Labels gestaltet werden können, um eine ansprechende Benutzeroberfläche zu erstellen. Der Inhalt wird durch die object-Eigenschaft `Content` definiert und kann damit beliebige Datentypen fassen.

Als Alternative kann das 'TextBlock'-Control verwendet werden, um spezifisch Text anzuzeigen. Im Vergleich zum 'Label' bietet der 'TextBlock' mehr Flexibilität und erweiterte Funktionen bei der Anzeige von Text. Insbesondere ist die Darstellungs-definierende Eigenschaft `Text` eine explizite string-Eigenschaft.

Generell kann man sagen, dass das 'Label'-Control gut geeignet ist, um einfache statische Texte anzuzeigen oder andere Steuerelemente zu beschriften. Der 'TextBlock' hingegen bietet mehr Flexibilität und erweiterte Funktionen, um dynamischen Text darzustellen und umfangreichere Textformatierungsoptionen zu ermöglichen. Die Wahl zwischen 'Label' und 'TextBlock' hängt also von den spezifischen Anforderungen und dem gewünschten Erscheinungsbild des Textes in der Anwendung ab.

Button

- Dient zum Auslösen eines Ereignisses

- XAML: `<Button></Button>`



- Wichtige Ereignisse:
 - `Click`: Mausklick
- Wichtige Eigenschaften:
 - `Background`: Hintergrundfarbe
 - `Foreground`: Vordergrundfarbe
 - `Content`: Beschriftung
 - `IsEnabled`: Verfügbarkeit des Buttons

Das Control 'Button' repräsentiert eine Schaltfläche, auf die der Benutzer klicken kann, um eine Aktion auszulösen. Es ist eines der grundlegenden Steuerelemente in WPF und bietet eine Vielzahl von Funktionen und Eigenschaften. Wichtige Eigenschaften:

1. `Content`: Die `Content`-Eigenschaft ermöglicht das Festlegen des Textes oder des Inhalts, der in der Schaltfläche angezeigt werden soll. Es kann entweder ein Textstring oder ein beliebiges anderes Steuerelement sein.
2. `Click`-Ereignis: Das `Click`-Ereignis tritt auf, wenn der Button geklickt wird. Es ermöglicht das Hinzufügen von Ereignishandlern, um spezifische Aktionen auszuführen, wenn der Button geklickt wird.
3. `IsEnabled`: Die `IsEnabled`-Eigenschaft steuert die Aktivierung des Buttons. Wenn sie auf "false" gesetzt wird, wird der Button ausgegraut und deaktiviert, und der Benutzer kann ihn nicht klicken.
4. `IsDefault`: Setzt den Button als den Standard-Button des aktuellen Bereichs (Fensters). D.h. wird die „Enter“-Taste gedrückt, während der Fokus in dem Bereich liegt, wird der Button betätigt.
5. `IsCancel`: Wie „IsDefault“, aber in Bezug auf den „ESC“-Button

Der 'Button' ermöglicht es Entwicklern, interaktive Benutzeroberflächen zu erstellen, indem sie Aktionen und Logik mit einem Klick auf die Schaltfläche verknüpfen. Mit seinen vielfältigen Eigenschaften und Ereignissen kann der Button an die Anforderungen und das gewünschte Erscheinungsbild der Anwendung angepasst werden.

Image

- Einfügen eines Bildes

- XAML: `<Image></Image>`



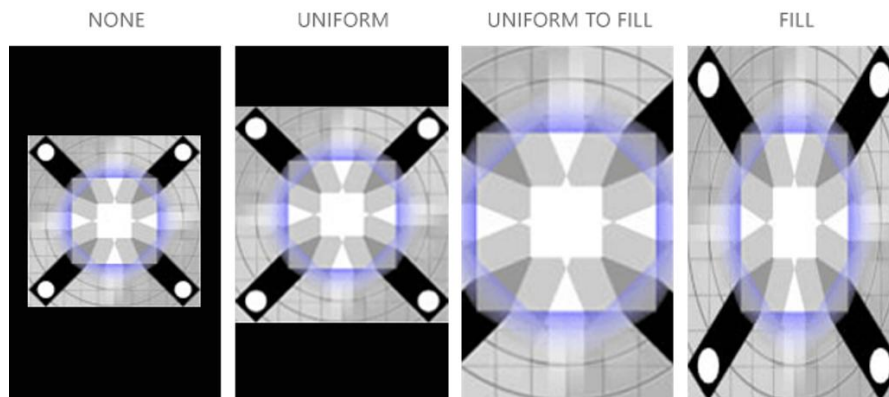
- Wichtige Eigenschaften:

- **Source:** Name und/oder Pfad des Bildes
- **Stretch:** Wie das Bild dargestellt wird
 - **Fill:** füllt das gesamte Image-Objekt, ohne Beachtung des Formates
 - **None:** fügt das Bild in Originalgröße ein
 - **Uniform:** vergrößert das Bild unter Beachtung des Formates und der Grenzen
 - **UniformToFill:** füllt das Image-Objekt unter Beachtung des Bildformates ohne Einhaltung der Grenzen

Das Control 'Image' ermöglicht das Anzeigen von Bildern in einer WPF-Anwendung. Es ist ein vielseitiges Steuerelement, das verschiedene Bildformate unterstützt und verschiedene Möglichkeiten bietet, Bilder anzuzeigen und zu manipulieren. Wichtige Eigenschaften:

1. **Source:** Die Source-Eigenschaft ermöglicht das Festlegen des Pfads oder der URI des Bildes, das angezeigt werden soll. Es kann sich um eine lokale Datei, eine Ressourcen-Bindung oder eine externe URL handeln.
2. **Stretch:** Die Stretch-Eigenschaft steuert, wie das Bild in der verfügbaren Fläche des Image-Controls skaliert und gestreckt wird. Es gibt verschiedene Optionen wie 'None' (keine Skalierung), 'Fill' (Füllen), 'Uniform' (proportionale Anpassung) und 'UniformToFill' (proportionale Anpassung mit Füllung).

Image - Skalierung



Die 'Stretch'-Eigenschaft des Image-Controls in WPF ermöglicht das Festlegen der Art und Weise, wie ein Bild in der verfügbaren Fläche des Image-Controls skaliert und gestreckt wird.

1. None: Das Bild wird nicht skaliert und behält seine ursprüngliche Größe bei. Es wird möglicherweise nicht vollständig angezeigt, wenn es größer ist als die verfügbare Fläche des Image-Control.
2. Fill: Das Bild wird gestreckt, um die gesamte verfügbare Fläche des Image-Controls auszufüllen, wobei das Seitenverhältnis des Bildes nicht beibehalten wird. Dies kann zu einer Verzerrung des Bildes führen, wenn das Seitenverhältnis des Image-Controls nicht mit dem des Bildes übereinstimmt.
3. Uniform: Das Bild wird proportional skaliert, um in die verfügbare Fläche des Image-Controls zu passen, wobei das Seitenverhältnis des Bildes beibehalten wird. Das gesamte Bild ist sichtbar, aber es kann Leerstellen geben, wenn das Seitenverhältnis des Image-Controls nicht mit dem des Bildes übereinstimmt.
4. UniformToFill: Das Bild wird proportional skaliert, um die gesamte verfügbare Fläche des Image-Controls auszufüllen, wobei das Seitenverhältnis des Bildes beibehalten wird. Das gesamte Image-Controls ist ausgefüllt, aber Teile des Bildes können abgeschnitten werden, wenn das Seitenverhältnis des Image-Controls nicht mit dem des Bildes übereinstimmt.

TextBox

- Bietet ein Textfeld zur Eingabe eines Textes



- XAML: `<TextBox></TextBox>`

- Wichtige Eigenschaften:

- `Background`: Hintergrundfarbe
- `Foreground`: Vordergrundfarbe
- `Text`: Inhalt des Textfeldes
- `IsEnabled`: Verfügbarkeit des Textfeldes
- `IsReadOnly`: Verfügbarkeit des Textfeldes und selektierbar
- `AcceptsReturn`: steuert das Verhalten der ENTER-Taste innerhalb der TextBox
- `AcceptsTab`: steuert das Verhalten der TAB-Taste innerhalb der TextBox
- `TextWrapping`: automatisches Einfügen von Zeilenumbrüchen

Das WPF-Control 'TextBox' ermöglicht die Eingabe und Anzeige von Text in einer WPF-Anwendung. Wichtige Eigenschaften:

1. `Text`: Die 'Text'-Eigenschaft ermöglicht den Zugriff auf den eingegebenen oder angezeigten Text im TextBox. Es kann sowohl lesbaren als auch beschreibbaren Text enthalten.
2. `IsReadOnly`: Die 'IsReadOnly'-Eigenschaft steuert, ob der Inhalt des TextBox bearbeitet werden kann oder nicht. Wenn sie auf 'true' gesetzt ist, kann der Benutzer den Text nicht ändern, sondern nur lesen bzw. markieren.
3. `AcceptsReturn`: Die 'AcceptsReturn'-Eigenschaft bestimmt, ob das TextBox Zeilenumbrüche akzeptiert, wenn die Eingabetaste gedrückt wird. Wenn sie auf 'true' gesetzt ist, kann der Benutzer durch Drücken von Enter einen Zeilenumbruch erzeugen.
4. `MaxLength`: Die 'MaxLength'-Eigenschaft legt die maximale Anzahl von Zeichen fest, die im TextBox eingegeben werden können. Wenn der Text die maximale Länge erreicht hat, kann der Benutzer keine weiteren Zeichen eingeben.
5. `TextWrapping`: Die 'TextWrapping'-Eigenschaft steuert, wie langer Text innerhalb des TextBox umgebrochen wird. Es gibt Optionen wie 'NoWrap' (kein Umbruch), 'Wrap' (Umbruch am Rand) und 'WrapWithOverflow' (Umbruch mit horizontaler Scrollleiste bei Bedarf).

ComboBox

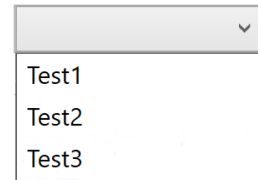
- Bietet die Möglichkeit aus mehreren Alternativen eine auszuwählen.

- XAML: `<ComboBox>`

```
<ComboBoxItem>Item</ComboBoxItem>
</ComboBox>
```

- Wichtige Eigenschaften:

- `Text`: Inhalt der ausgewählten Zeile
- `IsEnabled`: Verfügbarkeit der ComboBox
- `IsEditable`: Einträge in der ComboBox werden Editierbar
- `SelectedIndex`: Setzen/Abfragen des selektierten Index
- `SelectedItem`: Abfragen des selektierten Objekts



Das Control 'ComboBox' ist ein vielseitiges Steuerelement, das dem Benutzer eine Liste von Optionen zur Auswahl bietet. Es kombiniert (optional) eine Textbox mit einem Dropdown-Menü und ermöglicht es dem Benutzer, entweder einen Wert aus der Dropdown-Liste auszuwählen oder einen eigenen Wert einzugeben. Wichtige Eigenschaften:

1. `ItemsSource`: Die '`ItemsSource`'-Eigenschaft ermöglicht das Festlegen der Datenquelle für die Optionen, die im Dropdown-Menü angezeigt werden. Dies kann eine Liste von Objekten, eine Sammlung oder eine andere Datenquelle sein. Alternativ können die Objekte statisch in der Default-Property innerhalb von XAML gesetzt werden.
2. `SelectedItem`: Die '`SelectedItem`'-Eigenschaft enthält den ausgewählten Wert aus der Dropdown-Liste. Wenn der Benutzer einen Wert auswählt, wird dieser in der '`SelectedItem`'-Eigenschaft gespeichert.
3. `SelectedValue` und `SelectedValuePath`: Diese Eigenschaften ermöglichen die Verwendung von Datenbindung, um den ausgewählten Wert des ComboBoxes an eine Eigenschaft eines anderen Objekts zu binden.
4. `DisplayMemberPath`: Mit der '`DisplayMemberPath`'-Eigenschaft kann angegeben werden, welche Eigenschaft der Elemente in der Dropdown-Liste angezeigt wird.
5. `IsEditable`: Die '`IsEditable`'-Eigenschaft steuert, ob der Benutzer einen eigenen Wert in die Textbox eingeben kann. Wenn sie auf 'true' gesetzt ist, kann der Benutzer neben den vorhandenen Optionen auch einen eigenen Wert eingeben.

CheckBox

- Bietet die Möglichkeit eine Option auszuwählen

- XAML: `<CheckBox></CheckBox>`



- Wichtige Ereignisse:
 - `Checked`: CheckBox wurde ausgewählt
 - `Unchecked`: Auswahl wurde wieder aufgehoben
- Wichtige Eigenschaften:
 - `Content`: Beschriftung
 - `IsEnabled`: Verfügbarkeit der CheckBox
 - `IsChecked`: Gibt den Status der CheckBox zurück

Das Control 'CheckBox' ist ein Steuerelement, das dem Benutzer ermöglicht, zwischen zwei Zuständen zu wählen: aktiviert (angehakt) oder deaktiviert (nicht angehakt).

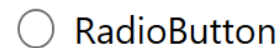
Wichtige Eigenschaften:

1. `IsChecked`: Die '`IsChecked`'-Eigenschaft gibt den Zustand des CheckBoxes an. Wenn sie auf '`true`' gesetzt ist, ist der CheckBox aktiviert (angehakt), andernfalls ist er deaktiviert (nicht angehakt).
2. `Content`: Die '`Content`'-Eigenschaft ermöglicht das Hinzufügen eines Textes oder eines anderen UI-Elements neben dem CheckBox, um den Benutzern weitere Informationen zur Option zu geben.
3. `IsThreeState`: Die '`IsThreeState`'-Eigenschaft steuert, ob in der CheckBox einen zusätzlicher dritter Zustand durch den Besitzer gesetzt werden kann. Dieser Zustand ('`null`') kann verwendet werden, um anzuzeigen, dass kein expliziter Zustand ausgewählt ist und durch das Programm auch unabhängig von dieser Eigenschaft gesetzt werden.

RadioButton

- Bietet die Möglichkeit aus mehreren Optionen eine auszuwählen

- XAML: `<RadioButton></RadioButton>`



- Wichtige Ereignisse:
 - `Checked`: RadioButton wurde ausgewählt
 - `Unchecked`: Auswahl wurde wieder aufgehoben
- Wichtige Eigenschaften:
 - `Content`: Beschriftung
 - `IsEnabled`: Verfügbarkeit des RadioButtons
 - `IsChecked`: Gibt den Status des RadioButtons zurück
 - `GroupName`: Gruppierung

Das Control 'RadioButton' ist ein Steuerelement, das dem Benutzer ermöglicht, eine Option aus einer Gruppe von Optionen auszuwählen. Hierbei kann nur ein RadioButton innerhalb derselben Gruppe gleichzeitig ausgewählt sein. Wichtige Eigenschaften:

1. `GroupName`: Die '`GroupName`'-Eigenschaft definiert die Zugehörigkeit des RadioButtons zu einer Gruppe von Optionen. Nur ein RadioButton innerhalb derselben '`GroupName`' kann zur gleichen Zeit ausgewählt sein. Alle RadioButtons mit derselben '`GroupName`' bilden eine logische Gruppe.
2. `IsChecked`: Die '`IsChecked`'-Eigenschaft gibt an, ob der RadioButton ausgewählt ist oder nicht. Wenn sie auf '`true`' gesetzt ist, ist der RadioButton ausgewählt, andernfalls ist er nicht ausgewählt.
3. `Content`: Die '`Content`'-Eigenschaft ermöglicht das Hinzufügen eines Textes oder eines anderen UI-Elements neben dem RadioButton, um den Benutzern weitere Informationen zur Option zu geben.

Es ist in WPF nicht möglich, ohne zusätzlichen C#-Code, zu prüfen, welcher RadioButton einer Gruppe ausgewählt ist. Hierfür müssen die RadioButtons einzeln überprüft werden.

Slider

- Ermöglicht eine Auswahl aus einem Zahlenbereich
- XAML: `<Slider></Slider>`
- Wichtige Eigenschaften:
 - `Minimum`: untere Grenze
 - `Maximum`: obere Grenze
 - `Value`: aktueller Wert
 - `Orientation`: Ausrichtung
 - `TickPlacement`: Platzierung von Markierungen
 - `TickFrequency`: Häufigkeit der Markierungen
 - `IsSnapToTickEnabled`: Ermöglicht die genaue Positionierung des Sliders

Das Control 'Slider' ist ein Steuerelement, das dem Benutzer ermöglicht, einen Wert innerhalb eines bestimmten Bereichs auszuwählen, indem er einen Schieberegler bewegt. Es stellt eine visuelle Darstellung eines Wertebereichs dar, von dem der Benutzer einen bestimmten Wert auswählen kann. Wichtige Eigenschaften:

1. `Minimum` und `Maximum`: Die Eigenschaften '`Minimum`' und '`Maximum`' definieren den Bereich der möglichen Werte, die der Benutzer auswählen kann. Der Schieberegler kann innerhalb dieses Bereichs verschoben werden.
2. `Value`: Die '`Value`'-Eigenschaft gibt den aktuellen ausgewählten Wert des Schiebereglers an. Wenn der Benutzer den Schieberegler verschiebt, wird der Wert entsprechend aktualisiert.
3. `Orientation`: Die '`Orientation`'-Eigenschaft bestimmt die Ausrichtung des Schiebereglers. Es kann horizontal oder vertikal sein, je nach Anforderungen des UI-Layouts.
4. `TickPlacement` und `TickFrequency`: Diese Eigenschaften ermöglichen es, Ticks auf dem Schieberegler anzuzeigen, um den Wertebereich visuell zu gliedern und dem Benutzer eine bessere Orientierung zu bieten. '`TickPlacement`' bestimmt die Position der Ticks (oben, unten, links oder rechts), während '`TickFrequency`' die Anzahl der Ticks angibt.
5. `IsSnapToTickEnabled`: Ist diese Eigenschaft auf '`True`' gesetzt, kann der Slider nur an den definierten Ticks platziert werden. Es ist nicht mehr möglich, Werte dazwischen auszuwählen. Dies funktioniert unabhängig davon, ob dem Benutzer die Ticks angezeigt werden oder nicht.

ProgressBar

- Stellt einen Prozessfortschritt mithilfe eines Balkens dar
- XAML: `<ProgressBar></ProgressBar>`
- Wichtige Eigenschaften:
 - `Minimum`: untere Grenze
 - `Maximum`: obere Grenze
 - `Value`: aktueller Wert
 - `IsIndeterminate`: Animation für einen unbekannten Prozessfortschritt

Das Control 'ProgressBar' ist ein Steuerelement, das dem Benutzer visuell den Fortschritt einer bestimmten Aufgabe oder den Status eines Vorgangs anzeigt. Es stellt eine horizontale oder vertikale Fortschrittsanzeige dar, die sich füllt, um den Fortschritt zu visualisieren. Wichtige Eigenschaften:

1. `Minimum` und `Maximum`: Die Eigenschaften 'Minimum' und 'Maximum' legen den Bereich der möglichen Werte für den Fortschritt fest. Der Fortschrittsbalken füllt sich entsprechend dem Verhältnis zwischen dem aktuellen Wert und dem Wertebereich.
2. `Value`: Die 'Value'-Eigenschaft gibt den aktuellen Fortschrittswert an. Der Wert kann zwischen dem Minimum- und dem Maximum-Wert liegen, die den gesamten Fortschrittsbereich definieren.
3. `Orientation`: Die 'Orientation'-Eigenschaft bestimmt die Ausrichtung des Fortschrittsbalkens. Es kann horizontal oder vertikal sein, je nach Anforderungen des UI-Layouts.
4. `IsIndeterminate`: Die 'IsIndeterminate'-Eigenschaft ermöglicht es, den Fortschrittsbalken in einen unbestimmten Modus zu versetzen, bei dem keine spezifische Fortschrittsanzeige angezeigt wird. Dies ist nützlich, wenn der genaue Fortschritt nicht bekannt ist.

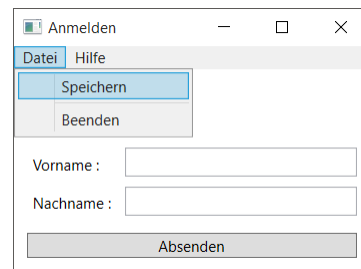
Menu

- Dient zum Anzeigen einer Menüleiste
 - Funktionsweise ähnelt dem Button-Element

- XAML:

```
<Menu x:Name="MyMenu">
  <MenuItem Header="Datei">
    <MenuItem Header="Speichern"/>
    <Separator/>
    <MenuItem Header="Beenden"/>
  </MenuItem>
</Menu>
```

- MenuItem's können auch in ContextMenus (Rechtsklick-Menüs) verwendet werden



Das Control 'Menu' ist ein Steuerelement, das die Darstellung eines Menüs ermöglicht, das dem Benutzer eine Auswahl von Befehlen oder Optionen bietet. Es stellt eine hierarchische Struktur von Menüelementen dar, die der Benutzer durch Klicken oder Überfahren mit der Maus auswählen kann.

Durch Einfügen von 'MenuItem'-Instanzen in die Default-Eigenschaft des Menüs können einzelne Einträge erzeugt werden. Jedes 'MenuItem' kann Text, Icon und weitere 'MenuItem'-Elemente enthalten, welche dann zu einem Untermenü werden.

Alternativ kann man das Ribbon-Control verwenden, welches eine moderne und effiziente Oberfläche für den Zugriff auf Befehle und Funktionen bietet. Im Gegensatz zum traditionellen Menü, das eine hierarchische Struktur von Menüelementen verwendet, bietet das Ribbon eine tabbasierte Navigation, visuelle Organisation von Befehlen und Funktionen in Gruppen und Schaltflächen, sowie direkten Zugriff auf häufig verwendete Funktionen über die Quick Access Toolbar. Das Ribbon ist hoch anpassbar und ermöglicht eine moderne Benutzeroberfläche mit ästhetischem Design, Symbolen und Tooltips. Es verbessert die Benutzererfahrung und ist optimal für Anwendungen mit umfangreichen Befehlen und Funktionen geeignet, insbesondere bei Touch-Input.

Listen

- Oft wird auch mit einer Liste von Daten gearbeitet
 - Basisklasse: `ItemsControl`
- Gängige Controls:
 - `ComboBox`
 - `ListBox`
 - `ListView`
 - Basiert auf `ListBox`
 - Modifizierbare View für komplexere Listen
 - `DataGrid`

In WPF gibt es verschiedene `ItemControls`, die dazu dienen, eine Liste von Elementen darzustellen und zu verwalten. Jedes `ItemControl` bietet eine einheitliche Möglichkeit, Datenquellen anzuzeigen und zu interagieren. Wichtige `ItemControls` und ihre Funktionen:

1. `ListBox`: Die `ListBox` unterstützt die Einzel- und Mehrfachauswahl von Elementen und bietet verschiedene Anpassungsmöglichkeiten für das Layout, das Erscheinungsbild und das Verhalten der Listenelemente. Die `ListBox` kann an eine Datenquelle gebunden werden und unterstützt die Darstellung komplexer Datenobjekte in einer vertikalen Auflistung.
2. `ComboBox`: siehe oben
3. `TreeView`: Der `TreeView` stellt eine hierarchische Struktur von Elementen in Form eines Baums darstellt. Es wird verwendet, um komplexe Datenstrukturen wie Verzeichnisbäume oder Kategorienhierarchien anzuzeigen. Der Baum kann erweitert und zusammengeklappt werden, um untergeordnete Elemente anzuzeigen oder zu verbergen.
4. `DataGrid`: Das `DataGrid` ist ein `ItemControl`, das eine tabellarische Darstellung von Daten ermöglicht. Es unterstützt die Bearbeitung von Zellen, die Sortierung, das Filtern und die Validierung von Daten. Es ist besonders nützlich, wenn es darum geht, große Datenmengen anzuzeigen und zu bearbeiten.

Fenster öffnen und schließen

```
//Öffnen eines neuen Fensters als gleichberechtigtes Fenster
new MainWindow().Show();

//Öffnen eines neuen Fensters als Dialogfenster mit Rückgabe des DialogResults
bool? dialogresult = new MainWindow().ShowDialog();

//Schließen des Fensters
this.Close();

//Beenden der Applikation
Application.Current.Shutdown();
```

14

Um in einer WPF-Anwendung ein Fenster zu öffnen, stehen verschiedene Methoden zur Verfügung. Nach der Instanziierung eines Fenster-Objekts kann es durch die Show()-Methode als gleichberechtigtes Fenster geöffnet werden.

Alternativ kann ein Fenster als Dialog geöffnet werden, indem die ShowDialog()-Methode aufgerufen wird. Dadurch wird das Fenster modal geöffnet, was bedeutet, dass die Benutzerinteraktion mit anderen Fenstern blockiert wird, bis das Dialogfenster geschlossen wird. Die ShowDialog()-Methode gibt einen boolschen Wert zurück, der im Dialog-Fenster durch die Eigenschaft ‚DialogResult‘ gesetzt werden kann.

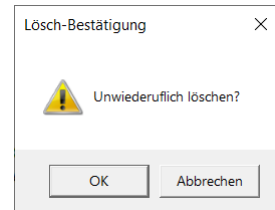
Das Hauptfenster einer WPF-Anwendung wird normalerweise automatisch geöffnet. Weitere Fenster können dann je nach Anwendungslogik geöffnet und geschlossen werden. Das Öffnen und Schließen von Fenstern ermöglicht es dir, die Benutzeroberfläche interaktiv zu gestalten und Benutzerinteraktionen zu ermöglichen.

Ein Fenster kann über die Close()-Methode geschlossen werden. Ist dies das letzte geöffnete Fenster der Applikation wird diese im Normalfall geschlossen. Alternativ kann die ganze Applikation über den Aufruf der Shutdown()-Methode der aktuellen App-Instanz geschlossen werden, auf die man über die statische Eigenschaft ‚Application.Current‘ zugreifen kann.

MessageBox

- Konfigurierbares Dialogfenster für kleine Benutzerabfragen

```
MessageBox.Show(  
    "Unwiederuflich löschen?",  
    "Löschen-Bestätigung",  
    MessageBoxButtons.OKCancel,  
    MessageBoxIcon.Exclamation  
)
```



- Gibt `MessageBoxResult` zurück

Die MessageBox ist ein nützliches Steuerelement, um dem Benutzer in Form von generischen Dialogfenstern Nachrichten anzuzeigen und Benutzerantworten zu erfassen.

Mit der Show-Methode kann eine MessageBox erstellt und angezeigt werden. Es können Text und Symbole verwendet werden, um den Nachrichtentyp zu kennzeichnen, sowie die angezeigten Buttons gewählt werden. Die Show-Methode gibt ein DialogResult zurück, der die Benutzeraktion (den angeklickten Button) repräsentiert

Es ist zu beachten, dass die MessageBox ein modales Fenster ist und die Ausführung des Codes blockiert. Für komplexe Dialoge oder benutzerdefinierte Nachrichten ist die Erstellung eines eigenen Fensters zu empfehlen. Die MessageBox verbessert die Benutzererfahrung, indem wichtige Informationen auf einfache und effektive Weise präsentiert werden.